

DISTRIB

Le smart contract DistribRegistry

expliqué — code, schémas et preuves sur Polygon Amoy

Version 1.0 – 21 août 2026

Réseau : Polygon Amoy (testnet) · Contrat vérifié · 19 tests automatisés

Document technique – complète docs/SPEC.md

1 · Pourquoi un smart contract, et pourquoi si peu de code

DistribRegistry n'est pas un coffre-fort : c'est un **registre d'états**. Il ne stocke ni fichier, ni argent — seulement des booléens et un compteur, par projet. Sa seule mission : rendre vérifiable, publiquement et pour toujours, qu'un ensemble de contributeurs a validé une version d'une œuvre à une date donnée.

Trois règles non négociables du projet gouvernent chaque ligne de ce contrat :

Non-négo #2 — Contrat minimal

Pas de gestion de fonds, pas de tokens, pas de paiement on-chain. Uniquement des états et des booléens. Surface strictement limitée à 5 fonctions — rien d'autre, pas même un getter supplémentaire.

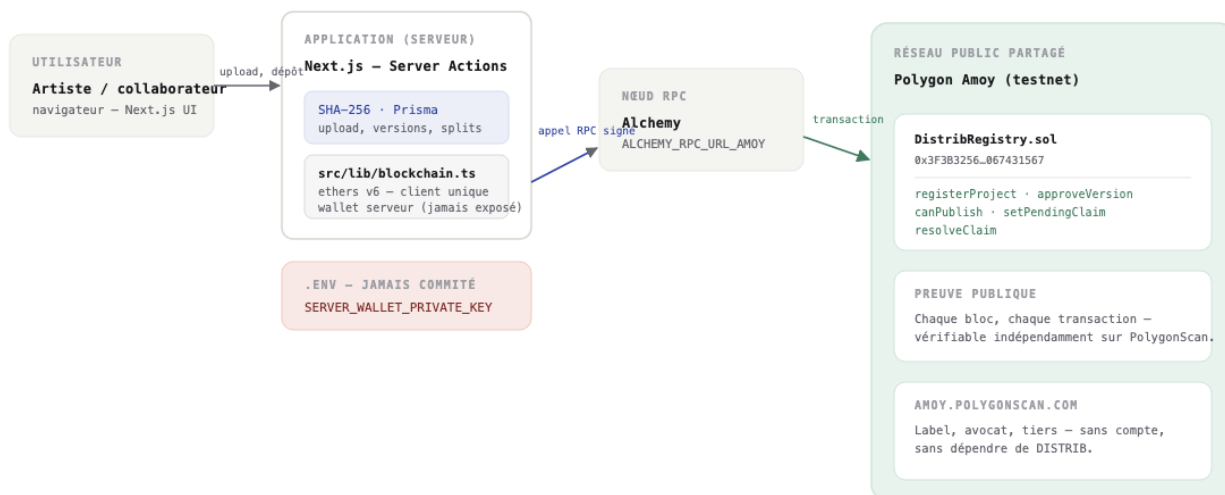
Non-négo #3 — 2 semaines de testnet obligatoires

Aucun déploiement mainnet avant deux semaines d'observation sur Amoy sans la moindre anomalie. Ce document documente précisément cette phase, en cours depuis le 18 août 2026.

Non-négo #5 — Blockchain invisible pour l'artiste

Un seul wallet — celui du serveur DISTRIB — signe toutes les transactions. L'artiste ne voit jamais un hash, une adresse ou un montant de gas.

Où se situe le contrat dans l'architecture



L'artiste ne signe jamais rien lui-même — seul le wallet serveur écrit sur la chaîne (non-négo #5).

Le serveur DISTRIB est le seul point de contact avec la blockchain. L'artiste passe uniquement par l'application ; le wallet privé ne quitte jamais le fichier .env du serveur.

2 · Le contrat, fonction par fonction

Code réel du fichier `contracts/contracts/DistribRegistry.sol`, Solidity 0.8.28, hérite d'`Ownable` et `ReentrancyGuard` (OpenZeppelin).

La structure de données — 3 champs, par projet

```
struct Project {
    bool exists;
    bool pendingClaim;
    uint256 approvedVersionCount;
}

mapping(bytes32 projectId => Project) private _projects;
mapping(bytes32 projectId => mapping(uint256 versionNumber => bytes32))
    private _versionHashes;
```

`projectId` est un `keccak256` du `cuid` généré côté serveur (Prisma) — le contrat ne connaît jamais l'identifiant lisible, seulement son empreinte. Le hash de chaque version approuvée est conservé séparément, indexé par numéro de version.

1. `registerProject` — enregistrer un projet, une seule fois

```
function registerProject(bytes32 projectId) external onlyOwner nonReentrant {
    if (_projects[projectId].exists) {
        revert ProjectAlreadyRegistered(projectId);
    }
    _projects[projectId].exists = true;
    emit ProjectRegistered(projectId);
}
```

Appelée une fois à la création d'un projet côté app. Un second appel sur le même identifiant échoue explicitement — impossible d'écraser un enregistrement existant.

2. `approveVersion` — ancrer une approbation unanime

```
function approveVersion(
    bytes32 projectId,
    uint256 versionNumber,
    bytes32 versionHash
) external onlyOwner nonReentrant {
    Project storage project = _projects[projectId];
    if (!project.exists) revert ProjectNotRegistered(projectId);
    if (versionHash == bytes32(0)) revert InvalidVersionHash();
    if (_versionHashes[projectId][versionNumber] != bytes32(0)) {
        revert VersionAlreadyApproved(projectId, versionNumber);
    }
    _versionHashes[projectId][versionNumber] = versionHash;
    unchecked { ++project.approvedVersionCount; }
    emit VersionApproved(projectId, versionNumber, versionHash);
}
```

`versionHash` = `keccak256` de la concaténation triée des SHA-256 de tous les fichiers de la version — trié pour être indépendant de l'ordre d'upload, donc recalculable par n'importe qui à partir des fichiers seuls. Une version déjà ancrée ne peut jamais être réécrite, même par le propriétaire : c'est la garantie d'immuabilité du contrat.

3. `canPublish` — la seule lecture publique

```
function canPublish(bytes32 projectId) external view returns (bool) {
    Project storage project = _projects[projectId];
    return
        project.exists &&
        project.approvedVersionCount > 0 &&
        !project.pendingClaim;
}
```

Recalculée à chaque appel — jamais un booléen stocké figé. C'est cette fonction que l'application interroge pour savoir si elle affiche le badge « Publiable ».

4 et 5. setPendingClaim / resolveClaim — le circuit des réclamations

```
function setPendingClaim(bytes32 projectId) external onlyOwner nonReentrant {
    Project storage project = _projects[projectId];
    if (!project.exists) revert ProjectNotRegistered(projectId);
    if (project.pendingClaim) revert ClaimAlreadyPending(projectId);
    project.pendingClaim = true;
    emit ClaimSet(projectId);
}

function resolveClaim(bytes32 projectId) external onlyOwner nonReentrant {
    Project storage project = _projects[projectId];
    if (!project.exists) revert ProjectNotRegistered(projectId);
    if (!project.pendingClaim) revert NoPendingClaim(projectId);
    project.pendingClaim = false;
    emit ClaimResolved(projectId);
}
```

Posées automatiquement par le pipeline de reconnaissance acoustique (Sprint 5) quand une correspondance dépasse le seuil de similarité. Tant que pendingClaim vaut vrai, canPublish renvoie faux — la distribution reste bloquée, quel que soit le nombre de versions déjà approuvées.

Sécurité : deux garde-fous OpenZeppelin, pas de code maison

Garde-fou	Rôle
Ownable	Seul le wallet serveur (owner) peut appeler les 4 fonctions d'écriture — un artiste n'a jamais accès direct.
ReentrancyGuard	Bloque les attaques de ré-entrance sur chaque fonction d'écriture — classe de faille historique sur Ethereum/Polygon.
Erreurs typées	6 erreurs personnalisées (ProjectAlreadyRegistered, VersionAlreadyApproved...) plutôt que des chaînes de caractères — moins de gas, messages explicites.

Aucun audit externe n'a été commandé — le budget du projet ne le permet pas. La seule ligne de défense est donc la simplicité : moins de 60 lignes de logique, 5 fonctions, aucune branche complexe. C'est un choix de conception assumé, pas un oubli.

3 · Ce qui se passe réellement à chaque dépôt

Deux transactions distinctes composent le parcours d'une version, dans cet ordre exact — la seconde ne peut jamais précéder la première dans l'application (voir `src/lib/vault.ts::finalizeSoloVersionIfReady`, qui a corrigé un bug réel : une version ne finalisait pas avant d'avoir au moins un fichier).

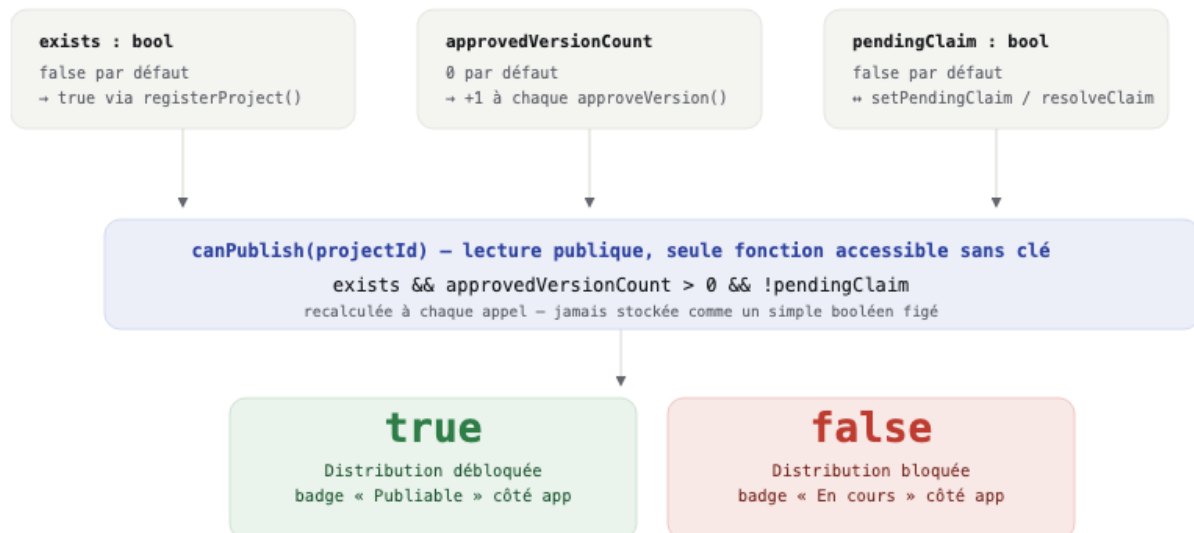


La phase 1 n'appelle **pas** le contrat — ajouter une fonction dédiée à l'ancrage par fichier aurait violé le principe de surface minimale. La preuve d'antériorité passe donc par une transaction brute du wallet serveur vers lui-même, le SHA-256 en donnée (*calldata*) : horodatée publiquement, sans toucher au contrat.

4 · La logique canPublish, visuellement

Trois champs suffisent à décider si un projet peut être distribué — aucun état cumulé, aucun flag caché ailleurs dans le contrat.

STRUCT PROJECT — 3 CHAMPS, RIEN D'AUTRE



Aucun autre champ, aucun getter additionnel — les 4 events (ProjectRegistered, VersionApproved, ClaimSet, ClaimResolved) sont la seule source d'audit historique, lisible par n'importe qui.

5 · Les 19 tests — ce qu'ils vérifient

Fichier `contracts/test/DistribRegistry.ts`, exécuté via `npx hardhat test` sur un réseau simulé local (aucun coût, aucune dépendance réseau). Relancés le 21 août 2026 pour ce document : **19 tests, 19 succès.**

```
DistribRegistry
  déploiement
    le déploreur (wallet serveur) est owner
  registerProject
    enregistre un projet et émet ProjectRegistered
    refuse le double enregistrement
    refuse un appelant non-owner
  approveVersion
    ancre une version et émet VersionApproved
    refuse un projet non enregistré
    refuse un hash nul
    refuse d'écraser une version déjà approuvée (immutabilité)
    accepte plusieurs versions successives
    refuse un appelant non-owner
  canPublish
    false pour un projet inconnu
    false sans version approuvée
    true après une version approuvée
    indépendant entre projets
  setPendingClaim / resolveClaim
    une réclamation bloque la publication, sa résolution la rétablit
    refuse une réclamation sur projet inconnu
    refuse une double réclamation
    refuse de résoudre sans réclamation en cours
    refusent les appelants non-owner
```

19 passing (69ms)

La philosophie de couverture

Catégorie	Exemple de test	Ce que ça garantit
Contrôle d'accès	« refuse un appelant non-owner » (×4)	Un artiste, même s'il connaît l'adresse du contrat, ne peut rien y écrire.
Immutabilité	« refuse d'écraser une version déjà approuvée »	Même le wallet serveur ne peut pas réécrire une preuve déjà ancrée.
Isolation	« indépendant entre projets »	Une réclamation ou une approbation sur un projet ne fuit jamais vers un autre.
Cycle complet	« une réclamation bloque... sa résolution la rétablit »	Le scénario réel de bout en bout, pas seulement les fonctions isolées.

Non-négo #3 rappelée : si un seul de ces 19 tests échouait, ou si un comportement inattendu apparaissait sur le testnet, le déploiement mainnet serait reporté sans exception.

6 · La preuve sur Polygon Amoy — pas une simulation

Le contrat est déployé et **vérifié** sur Polygon Amoy : n'importe qui peut lire son code source exact sur l'explorateur public, sans dépendre de la parole de DISTRIB.

Élément	Valeur
Adresse du contrat	0x3F3B3256467d4aab444e272237a2cc0067431567
Réseau	Polygon Amoy (testnet), chaîne 80002
Compilateur	Solidity v0.8.28+commit.7893614a, optimiseur 200 runs, EVM cancun
Licence déclarée	MIT
Explorateur	amoy.polygonscan.com/address/0x3F3B...1567

Le contrat vérifié — 8 transactions réelles

Amoy Testnet

Search by Address / Txn Hash / Block / Token

polygonscan

HomeBlockchainTokensNFTsAPIMore

Contract

0x3F3B3256467d4aab444e272237a2cc0067431567

Source Code

</> API

Overview

POL BALANCE
0 POL

More Info

CONTRACT CREATOR
0xED2a9C2c...DB4032556 | 2 days ago

Multichain Info

N/A

Transactions

Token Transfers (ERC-20)

Contract

Events

Latest 8 from a total of 8 transactions

Download Page Data

Transaction Hash	Method	Block	Age	From	To	Amount	Txn Fee
0xbd2b26ae78...	Approve Versi...	45189941	2 days ago	0xED2a9C2c...DB4032556	0x3F3B3256...067431567	0 POL	0.00277638
0x57c01c03a6...	Register Proje...	45189895	2 days ago	0xED2a9C2c...DB4032556	0x3F3B3256...067431567	0 POL	0.0018024
0x82f911f3f8d...	Approve Versi...	45188846	2 days ago	0xED2a9C2c...DB4032556	0x3F3B3256...067431567	0 POL	0.00277638
0x561abfc05c0...	Register Proje...	45188810	2 days ago	0xED2a9C2c...DB4032556	0x3F3B3256...067431567	0 POL	0.0018024
0x7faf5ecf7c95...	Approve Versi...	45188475	2 days ago	0xED2a9C2c...DB4032556	0x3F3B3256...067431567	0 POL	0.00416457
0x933b0a634b...	Register Proje...	45188393	2 days ago	0xED2a9C2c...DB4032556	0x3F3B3256...067431567	0 POL	0.0018024
0x7d72aa116b...	Approve Versi...	45188252	2 days ago	0xED2a9C2c...DB4032556	0x3F3B3256...067431567	0 POL	0.00874446
0x1081751455...	Register Proje...	45188222	2 days ago	0xED2a9C2c...DB4032556	0x3F3B3256...067431567	0 POL	0.0054072

Download CSV Export

A contract address hosts a small image

This website uses cookies to improve your experience. By continuing to use this website, you agree to its Terms and Privacy Policy. Got it!

Capture réelle, 21 août 2026. Le badge « Source Code » confirme la vérification : PolygonScan décode déjà les noms de fonctions (« Approve Versi... », « Register Proje... ») au lieu d'afficher du hexadécimal brut.

Notre code, tel qu'il apparaît sur l'explorateur

Amoy Testnet

Search by Address / Txn Hash / Block / Token

TransactionsToken Transfers (ERC-20)ContractEvents

CodeRead ContractWrite Contract

Source Code Verified

Exact Match

</> API

CONTRACT NAME

DistribRegistry

COMPILER VERSION

v0.8.28+commit.7893614a

OPTIMIZATION ENABLED

Yes with 200 runs

OTHER SETTINGS

cancun EvmVersion

LICENSE

MIT

Source

Similar Contracts

More Options

Contract Source Code (Solidity Standard Json-Input format)

EXPLORER

SEARCH

project/contracts/DistribRegistry.sol

OUTLINE

CONTRACT

DistribRegistry

FUNCTION

registerProject

approveVersion

canPublish

setPendingClaim

resolveClaim

Ownable.sol

DistribRegistry.sol

project > contracts > DistribRegistry.sol

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.28;
3
4 import {Ownable} from "@openzeppelin/contracts/access/Ownable.sol";
5 import {ReentrancyGuard} from "@openzeppelin/contracts/utils/ReentrancyGuard.sol";
6
7 /// @title DistribRegistry - registre d'états des projets musicaux DISTRIB
8 ///
9 /// NON-NÉGO #2 (doc master) : contrat minimal. Pas de gestion de fonds, pas de
10 /// tokens, pas de paiements on-chain - uniquement des états et des booléens.
11 /// Surface STRICTEMENT limitée à 5 fonctions : registerProject, approveVersion,
12 /// canPublish, setPendingClaim, resolveClaim. RIEN d'autre (pas même de getter
13 /// additionnel : les events sont la source d'audit publique).
14 ///
15 /// NON-NÉGO #5 : seul le wallet serveur DISTRIB (owner) écrit ici. Les artistes
16 /// n'interagissent jamais directement avec le contrat.
17 contract DistribRegistry is Ownable, ReentrancyGuard {
18     struct Project {
19         bool exists;
20         bool pendingClaim;
21         uint256 approvedVersionCount;
22     }
23
24     // projectId = keccak256(cuid du projet en base) - calculé côté serveur.
25     mapping(bytes32 projectId => Project) private _projects;
26     // Hash de version (keccak256 des SHA-256 de fichiers triés), par numéro.
27     mapping(bytes32 projectId => mapping(uint256 versionNumber => bytes32))
```

This website uses cookies to improve your experience. By continuing to use this website, you agree to its Terms and Privacy Policy.

Got it!

Contract ABI

</> API

Panneau « FUNCTION » à gauche : exactement 5 fonctions listées — *registerProject*, *approveVersion*, *canPublish*, *setPendingClaim*, *resolveClaim*. Aucune fonction cachée, vérifiable par quiconque sans compte ni accès particulier.

Le détail d'une transaction réelle

Amoy Testnet

Search by Address / Txn Hash / Block / Token

polygonscan

HomeBlockchainTokensNFTsAPIMore

Transaction Details

OverviewLogs (2)State

</> API

TRANSACTION ACTION

CallApprove Version

Function by 0xED2a9C2c...DB4032556 on 0x3F3B3256...067431567

[This is a POLY Amoy Testnet transaction only]

Transaction Hash:

0x82f911f3f8d4f16a6379cf0aebef27ed78558a2d908738cac096fe4afc21e599

Status:

Success

Block:

45188846257118 Block Confirmations

Timestamp:

2 days agoAug-17-2026 10:54:47 PM +UTC

From:

0xED2a9C2c3DD0BA7f28a92A824D0fcF2DB4032556

To:

0x3F3B3256467d4a44e272237a2cc0067431567

Value:

0 POL

Transaction Fee:

0.002776380005830398 POL

Gas Price:

30.000000063 Gwei (0.000000030000000063 POL)

More Details:

+ Click to show more

A transaction is a cryptographically signed instruction that changes the blockchain state. Block explorers track the details of all transactions in the network. Learn more about transactions in our Knowledge Base.

This website uses cookies to improve your experience. By continuing to use this website, you agree to its Terms and Privacy Policy.

Got it!

Powered by Polygon Blockchain

Back to Top

Transaction 0x82f911f3...21e599 — appel réel de approveVersion, statut Success, bloc 45188846, 17 août 2026. Frais réel : 0,00277638 POL (à ce prix du gas testnet, quelques millièmes de centime).

« Read Contract » — aucune lecture cachée

Amoy Testnet

Search by Address / Txn Hash / Block / Token

Transactions

Token Transfers (ERC-20)

Contract

Events

Code

Read Contract

Write Contract

Read Contract

Connect Wallet

Params

+ Expand All

Reset

1. canPublish (0xd5ea03d8) projectid

2. owner (0x8da5cb5b)

A contract address hosts a smart contract, which is a set of code stored on the blockchain that runs when predetermined conditions are met. Learn more about addresses in our [Knowledge Base](#).

Powered by Polygon PoS Chain

Back to Top

Add Polygon Amoy Network

This website uses cookies to improve your experience. By continuing to use this website, you agree to its [Terms](#) and [Privacy Policy](#).

Got it!

Released: Sep 6, 2024 (POLY AMOY A1) | Built by Team [Fibonacci](#)

[Terms & Privacy](#) | [About Us](#) | [Contact Us](#) | [Feedback](#)

L'onglet de lecture du contrat ne liste que 2 fonctions : `canPublish` (la nôtre) et `owner` (héritée d'`Ownable`). Confirmation directe, depuis l'explorateur, qu'aucun getter supplémentaire n'a été ajouté — conforme au non-négo #2.

Pour vérifier soi-même, sans rien installer : ouvrir

amoy.polygonscan.com/address/0x3F3B3256467d4aab444e272237a2cc0067431567 dans n'importe quel navigateur.

7 · Combien coûte une transaction

Estimation à partir de la consommation de gas **réellement mesurée** sur ces mêmes transactions Atoy, appliquée au prix du gas sur Polygon **mainnet** interrogé en direct le 19 août 2026 (~275 gwei, deux lectures indépendantes) et au cours du POL (~0,108 €).

Transaction	Gas consommé	Coût estimé (mainnet)
Ancrage d'un fichier (anchorFileHash)	24 160	~0,0007 €
Approbation de version (approveVersion)	92 546	~0,0028 €
Enregistrement d'un projet (registerProject)	~60 000	~0,0018 €

Un cycle complet (dépôt + approbation) revient à ~0,0035 € — les ~20 € de POL prévus par le budget du projet couvrent plusieurs milliers de transactions à ce niveau de prix. Le gas Polygon reste volatile : voir docs/SPEC.md pour la méthode complète et l'avertissement associé.

Annexe · Glossaire, en langage clair

Terme	Explication
Wallet serveur	Le compte qui signe toutes les transactions à la place de l'artiste. Sa clé privée vit uniquement dans le fichier .env du serveur, jamais exposée.
Transaction	Une action écrite pour toujours sur la blockchain. Chaque transaction a un identifiant unique (le hash) et coûte un peu de gas.
Bloc	Un paquet de transactions validées ensemble, environ toutes les 2 secondes sur Polygon. Chaque transaction appartient à exactement un bloc.
Gas / gwei	Le gas mesure le travail de calcul d'une transaction ; le gwei (milliardième de POL) en fixe le prix unitaire. Coût = gas × prix du gas.
Calldata	Les données brutes envoyées avec une transaction — ici, le SHA-256 d'un fichier, sans passer par une fonction du contrat.
keccak256	La fonction de hachage utilisée par Ethereum/Polygon — transforme n'importe quelle donnée en une empreinte de 32 octets, irréversible.
Ownable (OpenZeppelin)	Un patron de sécurité standard : une seule adresse (l'owner) peut appeler certaines fonctions. Ici, c'est toujours le wallet serveur.
Contrat vérifié	Le code source exact a été publié sur PolygonScan et recompilé pour prouver qu'il correspond au bytecode déployé — vérifiable par n'importe qui.